Design of Test Cases and a Maintenance Plan for a Sample Software Module

Case Study 3

Aim

To design test cases for a sample software module and prepare a maintenance plan that keeps the module reliable after release.

Objective

- Understand the requirements of the selected module.
- Design functional, boundary and negative test cases.
- Define test data, expected results and pass / fail criteria.
- Prepare a maintenance plan covering corrective, adaptive, perfective and preventive maintenance.

Selected Software Module

User Login Module of a web application. The user enters a username (registered e-mail) and a password. The system validates the credentials, grants access, and locks the account after 3 consecutive failed attempts.

Module Requirements

- R1: A valid user can log in and reach the dashboard.
- R2: Invalid credentials display an error message.
- R3: Username and password fields cannot be empty.
- R4: Password must be 8–16 characters.
- R5: Account is locked after 3 failed attempts.
- R6: A “Forgot Password” link sends a reset e-mail.

Test Design Techniques Used

- **Equivalence partitioning:** valid and invalid input classes.
- **Boundary value analysis:** password lengths 7, 8, 16 and 17.
- **Negative testing:** blank fields, SQL-injection strings.

Test Cases

ID	Scenario	Test Data	Expected Result	Req.
TC01	Valid login	user@mail.com / Pass@1234	Dashboard displayed	R1
TC02	Wrong password	user@mail.com / wrong123	Error: “Invalid credentials”	R2
TC03	Unregistered user	none@mail.com / Pass@1234	Error: “Invalid credentials”	R2
TC04	Blank username	(empty) / Pass@1234	Error: “Username required”	R3
TC05	Blank password	user@mail.com / (empty)	Error: “Password required”	R3
TC06	Password length 7 (below minimum)	7 characters	Validation error	R4
TC07	Password length 8 (minimum)	8 characters	Accepted	R4

ID	Scenario	Test Data	Expected Result	Req.
TC08	Password length 16 (maximum)	16 characters	Accepted	R4
TC09	Password length 17 (above maximum)	17 characters	Validation error	R4
TC10	3 consecutive wrong attempts	Wrong password × 3	Account locked message	R5
TC11	Login after lock with correct password	Valid credentials	Access denied until unlocked	R5
TC12	SQL injection attempt	' OR '1'='1	Rejected, no login	Security
TC13	Forgot password link	Registered e-mail	Reset e-mail sent	R6
TC14	Password masking	Type password	Characters shown as dots	Usability

Test Execution Summary (Template)

Total Test Cases	Executed	Passed	Failed	Blocked
14	14	Fill after execution	Fill after execution	0

Note: Pass / fail values are entered after running the tests on the actual application.

Defect Reporting Format

Field	Description
Defect ID	Unique identifier (e.g., BUG-101)
Summary	Short description of the problem
Steps to reproduce	Exact steps with test data
Severity / Priority	Critical, Major, Minor / High, Medium, Low
Status	New, Assigned, Fixed, Retested, Closed

Maintenance Plan

Type	Purpose	Example for Login Module
Corrective	Fix defects found after release	Fix a bug where the lock counter does not reset after a successful login
Adaptive	Adjust to changes in environment	Support a new browser version or OS update
Perfective	Improve features and performance	Add social login / two-factor authentication
Preventive	Avoid future problems	Refactor code, update libraries, add monitoring

Maintenance Schedule

Activity	Frequency	Responsible
Bug triage and fixes	Weekly	Developer / QA
Regression testing before each release	Every release	QA Engineer
Security patches and library updates	Monthly	Developer
Log and performance review	Monthly	DevOps
Backup verification	Quarterly	Administrator
Documentation update	Every release	Technical Writer / Developer

Results

- 14 test cases covering functional, boundary, negative and security scenarios were designed.
- Each test case is traceable to a requirement.
- A maintenance plan with schedule and responsibilities was prepared.

Conclusion

Systematic test design finds defects early, and a maintenance plan keeps the module secure and reliable after release. Together they reduce long-term cost and improve user trust in the software.